GIT & GITHUB BASICS

Learn How Professional Coders Work Together

- Save your code safely
- Work with friends on the same project
- Track every change you make
- Real skills used by professional programmers

What We'll Cover Today

Part 1 — The Basics

What Git and GitHub actually are, and why every programmer uses them

Part 2 — Key Concepts + Hands-On Practice

Cloning, branches, status, and pulling — with live practice for each

Part 3 — The Complete Workflow

The 4-step routine professionals use every single day

PART 1

The Basics

What Git and GitHub actually are — and why they matter

CONCEPT

What is Git?

- ✓ Saves different versions of your code — like save points in a video game
- ✓ Lets you work with friends without losing anyone's work
- ✓ Lets you go back in time if you make a mistake
- ✓ Keeps everything organized in one place

💡 Think of it like...

- *Google Docs, but for code*
- *A time machine for your files*
- *A filing system that never loses anything*

CONCEPT

What is GitHub?

- ✓ Stores your code on the internet
- ✓ Lets you share projects with your team
- ✓ Shows who changed what, and when
- ✓ Syncs code between different computers

💡 Think of it like...

- *Google Drive, but for code*
- *A central meeting place for your project*
- *A backup that's always safe in the cloud*

Git vs GitHub: What's the Difference?

Simple way to remember: Git = the tool. GitHub = the place to store it.

	GIT	GITHUB
What is it?	Software on your computer	Website in the cloud
What it does	Tracks changes to files	Stores and shares files
Where it lives	Your computer	Internet servers
Who uses it	You (locally)	You + your team

Why Learn Git & GitHub?

- ✓ You never lose your work — everything is saved
- ✓ You can work with friends without conflicts
- ✓ You can undo mistakes instantly
- ✓ Your teacher can see your progress
- ✓ It's a skill employers look for

REAL COMPANIES THAT USE GITHUB

Google | Microsoft | Apple | Facebook | Netflix | SpaceX

Learning Git now = a big advantage in your coding future!

PART 2

Key Concepts + Hands-On Practice

Cloning, branches, status, and pulling — with live practice for each

CONCEPT 1 OF 4

What is Cloning?

- ✓ Cloning = making a copy of a project from the internet to your computer
- ✓ Cloning is STATIC — it copies files at one moment in time
- ✓ It does NOT update automatically
- ✓ To get new updates later, you'll need to PULL (coming up soon)

💡 Real Example

- *Your teacher puts a project on GitHub*
- *You clone it — you now have a copy*
- *Two weeks later, teacher adds new files*
- *Your computer still has the OLD version until you pull*

PRACTICE 1 OF 4

Clone a Repository

🎯 *Copy a real project from GitHub to your computer*

1. Open your terminal (Mac: Terminal app, Windows: Command Prompt or Git Bash, Chromebook: web IDE terminal)

2. Go to your Projects folder

```
$ cd ~/Desktop/Projects
```

3. Clone the repository — ask your teacher for the link

```
$ git clone [your-teacher's-link]
```

4. Enter the folder

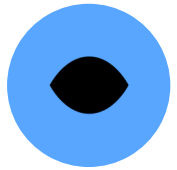
```
$ cd git-practice-students
```

5. See what you got

```
$ ls
```

WHAT TO EXPECT

Clone Results



In your terminal, you should see:

```
Cloning into 'git-practice-students'...
remote: Counting objects: 100% (42/42)
Receiving objects: 100% (42/42), 5.67 KiB
Unpacking objects: 100% (42/42), done.
```

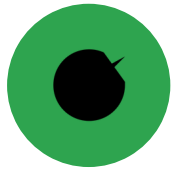
When you run ls, you'll see:

```
README.md
students.txt
examples/
ACTIVITIES.md
```

💡 *A new folder named git-practice-students now exists on your computer — that's your local copy!*

SUCCESS CHECK

Clone Success Checklist



- ✓ Opened the terminal / command prompt
- ✓ Navigated to your Projects folder
- ✓ Ran the clone command
- ✓ Saw the "Cloning into..." message
- ✓ Saw a new folder appear on your computer
- ✓ Entered the folder with cd
- ✓ Saw the files inside with ls or dir

ALL CHECKED?

You're ready for the next step!

IF SOMETHING WENT WRONG:

- Check the GitHub link is correct
- Make sure Git is installed
- Ask your teacher for help

CONCEPT 2 OF 4

What is a Branch?

- ✓ A branch is a separate copy of your project where you can experiment safely
- ✓ Your experiments don't break the main project
- ✓ You can work on different features at the same time
- ✓ Friends can work on main while you work on yours

💡 Real Example — Building a Video Game

- *main = the working game everyone plays*
- *feature/jumping = your branch adding a jump mechanic*
- *feature/menu = a friend's branch adding a menu*
- *Both work at the same time without breaking anything!*

PRACTICE 2 OF 4

Create Your Own Branch

🎯 *Create a safe space to experiment*

1. Check what branch you're on

```
$ git status
```

2. Create a new branch — replace [your-name] with your actual name

```
$ git checkout -b feature/[your-name]
```

3. Confirm you switched

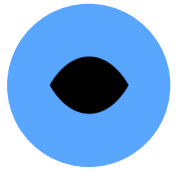
```
$ git status
```

4. See all your branches

```
$ git branch
```

WHAT TO EXPECT

Branch Creation Output



After git status (before):

```
On branch main
nothing to commit, working tree clean
```

After creating your branch:

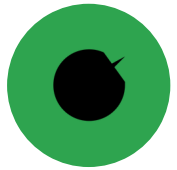
```
Switched to a new branch
'feature/alex-smith'

# git branch shows:
* feature/alex-smith
```

💡 *The asterisk (*) next to a branch name in "git branch" shows which branch you're currently on!*

SUCCESS CHECK

Branch Creation Checklist



- [x] Ran git status and saw you were on main
- [x] Created a new branch with git checkout -b
- [x] Used a good branch name with your name
- [x] Ran git status and saw your new branch
- [x] Ran git branch and saw both branches
- [x] Understand branches are separate and safe

ALL CHECKED?

You're ready for the next step!

IF SOMETHING WENT WRONG:

- "Branch already exists?" → use a different name
- "Not a git repository?" → make sure you're in the project folder
- "Git not found?" → make sure Git is installed

CONCEPT 3 OF 4

What is Git Status?

- ✓ Git Status is your compass — it tells you exactly where you are
- ✓ Shows what branch you're on right now
- ✓ Shows what files you changed, and what's new
- ✓ It NEVER changes anything — completely safe to run anytime

💡 When To Use It

- *When you're confused about where you are*
- *Before switching branches*
- *Before pulling updates*
- *Whenever you want to check your progress*

PRACTICE 3 OF 4

Use Git Status

🎯 *Learn to read what Git tells you*

1. Make sure you're on your branch

```
$ git status
```

2. Create a new file

```
$ echo "This is my work" > my-file.txt
```

3. Check your status again — notice the change!

```
$ git status
```

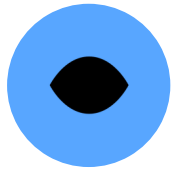
4. Edit the file (open it, change the text, save)

5. Check status one more time

```
$ git status
```

WHAT TO EXPECT

Git Status Output



After creating a new file:

```
On branch feature/alex-smith
Untracked files:
  my-file.txt
```

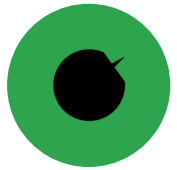
After editing the file:

```
On branch feature/alex-smith
Changes not staged for commit:
  modified: my-file.txt
```

💡 *Key takeaway: git status is your best friend. Use it constantly — before, during, and after everything you do.*

SUCCESS CHECK

Git Status Checklist



- [x] Ran git status on your branch
- [x] Created a new file
- [x] Saw the file show as "Untracked"
- [x] Edited the file
- [x] Saw the file show as "modified"
- [x] Understand what each message means

ALL CHECKED?

You're ready for the next step!

IF SOMETHING WENT WRONG:

- Nothing showing up? Make sure you saved the file
- Wrong branch? Run git checkout [branch-name] first
- Still confused? Just run git status again — it's always safe

CONCEPT 4 OF 4

What is Git Pull?

- ✓ Pull = download the latest updates from GitHub to your computer
- ✓ Use it when your friends pushed new code
- ✓ Use it when your teacher added new files
- ✓ Important: Pull only works on main — your branch stays separate and safe

💡 When Do You Use It?

- *Your friends pushed new code*
- *Your teacher added new files*
- *You want the newest version*
- *You need to stay in sync with the team*

PRACTICE 4 OF 4

Pull Updates from Main

🎯 *Your teacher adds a new file to main on GitHub — let's go get it*

1. Check what branch you're on

```
$ git status
```

2. Switch to main

```
$ git checkout main
```

3. Pull the latest updates

```
$ git pull origin main
```

4. See the new files

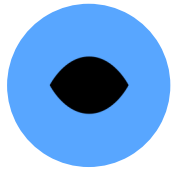
```
$ ls
```

5. Switch back to your branch

```
$ git checkout feature/[your-name]
```

WHAT TO EXPECT

Git Pull Output



When you pull (if there are updates):

```
remote: Counting objects: 100%
Updating abc1234..def5678
Fast-forward
new-challenge.txt | 10 +++++
```

Or, if nothing changed yet:

```
Already up to date.
```

💡 *Notice: new files from main are NOT automatically on your branch — that separation is exactly what keeps your work safe.*

PART 3

The Complete Workflow

The 4-step routine professionals use every single day

The Complete Workflow

When you want to get updates without losing your work — this is the most important skill you'll learn!

1. Stash

Save your work temporarily

2. Checkout Main

Switch to the official version

3. Pull

Download latest updates

4. Return & Pop

Go back to your branch and get your work

STEP 1 OF 4

Stash — Save Your Work

What it does: Puts your changes in a temporary safe box.

Why: You have unsaved changes and need to switch branches without losing them.

TYPE THIS

```
$ git stash
```

YOU'LL SEE

```
Saved working directory
and index state WIP on
feature/alex-smith
```

STEP 2 OF 4

Checkout Main — Switch Branches

What it does: Switches your workspace to the official project.

Why: You need to be on main to get updates, and your branch stays safe while you're away from it.

TYPE THIS

```
$ git checkout main
$ git status
$ git branch
```

YOU'LL SEE

```
Switched to branch 'main'
```

STEP 3 OF 4

Pull — Get Latest Updates

What it does: Downloads the newest code from GitHub.

Why: Your team added new features or fixed bugs, and you need the latest version.

TYPE THIS

```
$ git pull origin main
$ ls
```

YOU'LL SEE

```
Already up to date.

# -- or --
Updating abc1234..def5678
Fast-forward
```

STEP 4 OF 4

Return & Pop — Get Your Work Back

What it does: Switches back to your branch and restores your stashed changes.

Why: Now you have the latest updates on main, and you want to keep working right where you left off.

TYPE THIS

```
$ git checkout feature/[your-name]
$ git stash pop
$ git status
```

YOU'LL SEE

```
Switched to branch
'feature/alex-smith'

On branch feature/alex-smith
Changes not staged for commit:
  modified: my-file.txt
```

Your Command Cheat Sheet

Keep this slide — you'll use these commands constantly.

`git clone [url]`	Copy a project to your computer
`git status`	See what's changed right now
`git checkout -b [name]`	Create and switch to a new branch
`git checkout [name]`	Switch to an existing branch
`git branch`	List all your branches
`git pull origin main`	Download the latest updates
`git stash`	Temporarily save uncommitted work
`git stash pop`	Bring your stashed work back
`git add [file]`	Mark a file ready to save
`git commit -m "msg"`	Save your changes with a message

Common Mistakes (And Easy Fixes)

`"fatal: not a git repository"`

Fix: You're in the wrong folder — use cd to go into your project folder first

`"branch already exists"`

Fix: Someone already used that branch name — pick a different, more specific one

`Forgot to save work before switching branches`

Fix: Run git stash first — it's always safe and reversible

`"git: command not found"`

Fix: Git isn't installed yet — ask your teacher or check the install guide

You Did It! 🎉

You now know more Git & GitHub than most beginners ever learn in their first month.

- Practice these commands on your own project this week
- Keep your cheat sheet handy — everyone looks things up, always
- Ask questions anytime — every professional was a beginner once